

Adaptive Linear Programming Decoding

Mohammad H. Taghavi and Paul H. Siegel

ECE Department, University of California, San Diego

Email: (mtaghavi, psiegel)@ucsd.edu

Abstract—The ability of linear programming (LP) decoding to detect failures, and its potential for improvement by the addition of new constraints, motivates the use of an adaptive approach in selecting the constraints for the underlying LP problem. In this paper, we show that the application of such adaptive methods can significantly reduce the complexity of the LP decoding algorithm, which, in the standard formulation, is exponential in the maximum row weight of the parity-check matrix. We further show that adaptively adding new constraints, e.g. by combining parity checks, can provide large gains in LP decoder performance.

I. INTRODUCTION

Linear programming (LP) decoding, as an approximation to maximum-likelihood (ML) decoding, was proposed by Feldman *et al.* [1]. Many observations suggest similarities between the performance of LP and iterative message-passing methods, e.g. in [2]. For example, we know that the existence of low-weight pseudo-codewords degrades the performance of both methods ([3], [1]). Therefore, it is reasonable to make use of the simpler geometrical structure of LP decoding to make predictions on the performance of message-passing algorithms.

On the other hand, there are differences which prevent us from making an explicit connection between these two approaches. For instance, given an LDPC code, adding additional parity checks that are satisfied by all the codewords can only improve LP decoding, while with message-passing algorithms, these parity checks may have a negative effect by introducing short cycles in the Tanner graph. This property of LP decoding allows improvements by tightening the relaxation. Another characteristic of LP decoding (the *ML certificate property*) is that its failure to find the ML codeword is detectable. More specifically, the decoder always gives either the ML codeword, or a nonintegral pseudo-codeword as the solution.

These two properties motivate the use of an adaptive approach in LP decoding which can be summarized as follows: Given a set of constraints that describe a code, start the LP decoding with a few of them, and sequentially and adaptively add more of the constraints to the problem until either the ML codeword is found or no further “useful” constraint exists. The goal of this paper is to explore the potential of this idea for LP decoding. We show that by putting LP in an adaptive setting, we can obtain the same performance as if a huge number of constraints were added to the relaxation from the beginning. In particular, let n , m , d_c^{max} , and d_v^{max} respectively denote

the length, the number of parity checks, the maximum check node degree, and the maximum variable node degree of the code. We have observed that the adaptive method generally converges with at most $O(m)$ constraints. This is a significant improvement compared to the standard description of LP decoding by Feldman *et al.* with $O(n + m2^{d_c^{max}})$ constraints and also their alternative high-density code polytope with $O(mn + m(d_c^{max})^2 + nd_v^{max}d_v^{max})$ constraints.

The rest of this paper is organized as follows. In Section II, we review Feldman’s LP decoding. In Section III, we introduce and analyze an adaptive algorithm to solve the original LP problem more efficiently. In Section IV, we study how adaptively adding additional constraints can improve the performance. Section V concludes the paper. Some of the proofs are omitted due to space limitations, and the reader is referred to a forthcoming paper for more details [4].

II. LP RELAXATION OF ML DECODING

Consider a binary linear code $\mathcal{C}$ of length n . If a codeword $\underline{y} \in \mathcal{C}$ is transmitted through a binary-input memoryless channel, the ML codeword given the received vector $\underline{r} \in \mathbb{R}^n$ is the solution to the optimization problem

$$\begin{aligned} &\text{minimize} && \underline{\gamma}^T \underline{x} \\ &\text{subject to} && \underline{x} \in \mathcal{C}, \end{aligned} \quad (1)$$

where $\underline{\gamma}$ is the vector of log-likelihood ratios defined as

$$\gamma_i = \log \left(\frac{\Pr(r_i | y_i = 0)}{\Pr(r_i | y_i = 1)} \right). \quad (2)$$

As an approximation to ML decoding, Feldman *et al.* proposed a relaxed version of this problem by first considering the convex hull of the local codewords defined by each row of the parity-check matrix, and then intersecting them to obtain what is called the *fundamental polytope* by Koetter *et al.* [3]. This polytope has a number of integral and nonintegral vertices, but the integral vertices exactly correspond to the codewords of $\mathcal{C}$. Therefore, whenever LP gives an integral solution, it is guaranteed to be the ML codeword.

In Feldman’s relaxation of the decoding problem, the following is done for each row $j = 1, \dots, m$ of the parity-check matrix. Suppose that the j th check node has the neighborhood set $N \subset \{1, 2, \dots, n\}$, i.e. N contains the indices of the variable nodes that are directly connected to this check node. Then, add the following constraints to the problem:

$$\sum_{i \in V} x_i - \sum_{i \in N \setminus V} x_i \leq |V| - 1, \quad \forall V \subset N \text{ such that } |V| \text{ is odd.} \quad (3)$$

This work was supported by the Center for Magnetic Recording Research, University of California, San Diego.

Throughout the paper, we refer to the constraints of this form as *parity-check constraints*. In addition, for any element x_i of the optimization variable, $\underline{x}$, the constraint that $0 \leq x_i \leq 1$ is also added.

III. ADAPTIVE LP DECODING

As any odd-sized subset V of the neighborhood N of each check node introduces a unique parity-check constraint, there are 2^{d_c-1} constraints corresponding to each check node of degree d_c . Therefore, the total number of constraints and, hence, the complexity of the problem, is exponential in terms of the maximum check node degree, d_c^{max} . This becomes more significant in a high density code where d_c^{max} increases with the code length, n . In this section, we show that LP relaxation of linear codes has some properties that allow us to solve the optimization by using a much smaller number of constraints.

A. Properties of the Relaxation Constraints

Definition 1: Given a constraint of the form

$$\underline{a}_i^T \underline{x} \leq b_i, \quad (4)$$

and a vector $\underline{x}_0 \in \mathbb{R}^n$, we call (4) an *active constraint* at $\underline{x}_0$ if

$$\underline{a}_i^T \underline{x}_0 = b_i, \quad (5)$$

and a *violated constraint* or, equivalently, a *cut* at $\underline{x}_0$ if

$$\underline{a}_i^T \underline{x}_0 > b_i. \quad (6)$$

Considering a constraint that generates a cut

$$\sum_{i \in V} x_i - \sum_{i \in N \setminus V} x_i > |V| - 1, \quad (7)$$

at point $\underline{x}$, we immediately make the following observations:

$$|V| - 1 < \sum_{i \in V} x_i \leq |V|, \quad (8)$$

$$0 \leq \sum_{i \in N \setminus V} x_i < x_j \quad \forall j \in V. \quad (9)$$

The following theorem reveals a special property of the constraints of the LP decoding problem.

Theorem 1: At any given point $\underline{x} \in [0, 1]^n$, at most one of the constraints introduced by each check node can be a cut. (Proof omitted.)

Having a linear (n, k) code with $m = n - k$ parity checks, a natural question is how we can find all the cuts defined by the LP relaxation at any given point $\underline{x} \in \mathbb{R}^n$. For any check node and an odd subset V of its neighborhood that introduces a cut, we know from (9) that the members of V are the variable nodes with the largest values among the neighbors of the check node. Therefore, sorting the elements of $\underline{x}$ before searching for a cut can simplify the procedure.

Consider a check node j . Without loss of generality, assume that variable nodes $1, 2, \dots, |N|$, are the neighbors of this check node, and they are sorted with respect to their values such that $x_1 \geq x_2 \geq \dots \geq x_{|N|}$. The following algorithm is an efficient way to find the cut generated by this check node at $\underline{x}$, if it exists.

Algorithm 1:

Step 1: Set $v = 1$, $V = \{1\}$ and $V^c \triangleq N \setminus V = \{2, 3, \dots, |N|\}$.

Step 2: Check the constraint (3). If it is violated, we have found the cut. Exit.

Step 3: Set $v = v + 2$. If $v \leq |N|$, move x_{v-1} and x_v (the two largest members of V^c) from V^c to V .

Step 4: If $v \leq |N|$ and (8) is satisfied, go to Step 2; otherwise, the check node does not provide a cut at $\underline{x}$.

If redundant calculations are avoided, this algorithm can find the cut generated by the check node, if it exists, in $O(d_c)$ time, where $d_c = |N|$ is the degree of the check node. Repeating the procedure for each check node, and considering $O(n \log n)$ complexity for sorting $\underline{x}$, the time required to find all the cuts at point $\underline{x}$ becomes $O(md_c^{max} + n \log n)$ ¹.

B. The Adaptive Procedure

The Simplex LP algorithm starts from a vertex of the problem polytope and visits different vertices of the polytope by traveling through the edges until it finds the optimum vertex. The time required to find the solution is approximately proportional to the number of vertices that have been visited, and this, in turn, is determined by the number and properties of the constraints in the problem. Hence, if we eliminate some of the intermediate vertices and only keep those which are close to the optimum point, we can reduce the complexity of the algorithm. To implement this idea in the adaptive LP decoding scheme, we run the LP solver with a minimal number of constraints to ensure boundedness of the solution, and depending on the LP solution, we add only “the useful constraints” that cut the current solution from the feasible region. This procedure is repeated until no further cut exists.

To start the procedure, we need at least n constraints so that the problem has a vertex that can become the solution of the first round. Using the condition that $0 \leq x_i \leq 1$, we add one side of these inequalities for each i , depending on whether increasing x_i increases or decreases the objective function. In other words, for each $i \in \{1, 2, \dots, n\}$, we initially have the constraint

$$\begin{aligned} 0 &\leq x_i && \text{if } \gamma_i > 0, \\ x_i &\leq 1 && \text{if } \gamma_i < 0. \end{aligned} \quad (10)$$

The optimum (and only) vertex of this initial problem corresponds to the result of (uncoded) hard decision based on the received vector. Now we proceed with the following algorithm:

Algorithm 2:

Step 1: Setup the initial problem according to (10).

Step 2: Run the LP solver.

Step 3: Search for all the cuts for the current solution.

Step 4: If one or more cuts are found, add them to the problem constraints and go to Step 2.

Claim 1: If at any iteration of Algorithm 2 no cut is found, the current solution is the solution of the LP decoder with all the relaxation constraints given in Section II.

¹For low density codes, it is better to sort the neighbors of each check node separately, so the total complexity becomes $O(md_c^{max} + md_c^{max} \log d_c^{max})$.

Proof: The claim follows from the fact that if at any stage no cut is found, the current solution is in the fundamental polytope. ■

C. A Bound on the Complexity

Theorem 2: The adaptive algorithm (Algorithm 2) converges with at most n iterations.

Proof: The final solution is a vertex $\underline{x}_f$ of the problem space determined by the initial constraints along with those added by the adaptive algorithm. Therefore, we can find n such constraints, $\kappa_i : \underline{\alpha}_i^T \underline{x} \leq \beta_i$, $i = 1, 2, \dots, n$, whose corresponding hyperplanes uniquely determine this vertex. This means that if we set up an LP problem with only those n constraints, the optimal point will be $\underline{x}_f$. Now, consider the k th intermediate solution, $\underline{x}_k$, that is cut off at the end of the k th iteration. At least one of the constraints, $\kappa_1, \dots, \kappa_n$, should be violated by $\underline{x}_k$, otherwise since $\underline{x}_k$ has a lower cost than $\underline{x}_f$, $\underline{x}_k$ would be the solution of LP with these n constraints. But we know that the cuts added at the k th iteration are all the possible constraints that are violated at $\underline{x}_k$. Consequently, at least one of the cuts added at each iteration should be among $\{\kappa_i\}$; hence, the number of iterations is at most n . ■

Remark 1: This theorem applies to any general LP problem where there is a fixed set of constraints, and at each iteration we add all the cuts generated by this set of constraints.

Corollary 1: The adaptive algorithm has at most $n(m+1)$ constraints at the final iteration.

Proof: Follows from Theorem 1 and Theorem 2. ■

For high-density codes of fixed rate, this bound guarantees convergence with $O(n^2)$ constraints, whereas the standard LP and the polytope given in [1] for high-density codes respectively require exponential and $O(n^3)$ constraints.

D. Numerical Results

To observe the complexity reduction due to the adaptive approach for LP decoding, we have performed simulations over random regular LDPC codes of various lengths, degrees, and rates on the AWGN channel. All the experiments were performed with the low SNR value of -1.0 dB, since in the high SNR regime the received vector is likely to be close to a codeword, in which case the algorithm converges fast, rather than demonstrating its worst-case behavior.

In the first scenario, we studied the effect of changing the check node degree d_c from 4 to 40 while keeping the code length at $n = 360$ and the rate at $R = \frac{1}{2}$. The simulation was performed over 400 blocks for each value of d_c . The maximum (average) number of iterations required to converge started from around 30 (14.5) for $d_c = 4$, and decreased monotonically down to 9 (5.9) for $d_c = 40$. The average and maximum numbers of parity-check constraints in the final iteration of the algorithm are plotted in Fig. 1. It is observed that both the average and the maximum values are almost constant, and remain below 270 for all the values of d_c . For comparison, the total number of constraints required for the standard (non-adaptive) LP decoding problem, which is equal to 2^{d_c-1} is also included in this figure. The decrease in the

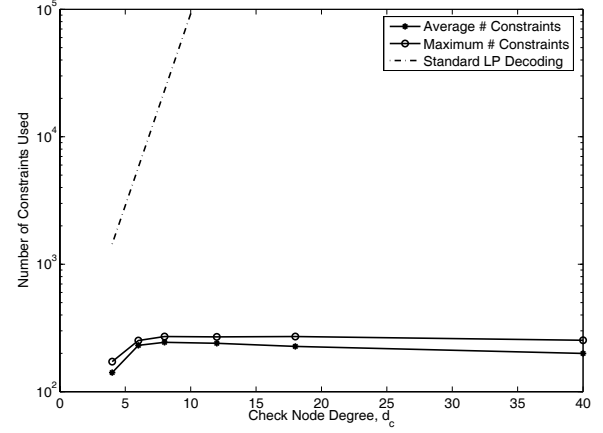


Fig. 1. The average and maximum number of parity-check constraints used versus check node degree, d_c , for fixed length $n = 360$ and rate $R = \frac{1}{2}$.

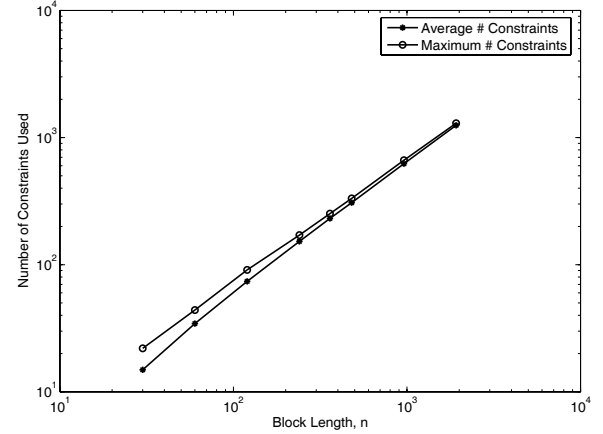


Fig. 2. The average and maximum number of parity-check constraints used versus block length, n , for fixed rate $R = \frac{1}{2}$ and check node degree $d_c = 6$.

number of required constraints translates to a large gain for the adaptive algorithm in terms of the running time. This gain increases exponentially with d_c , so that with the LP solver that we have used in our work (GLPK [5]), the adaptive algorithm always converges several thousand times faster than standard LP for $d_c = 8$.

In the second case, we studied random (3,6) codes of lengths $n = 30$ to $n = 1920$. For all values of n , the maximum (average) number of required iterations remained between 10 and 16 (5 and 11). The average and maximum numbers of parity-check constraints in the final iteration are plotted versus n in Fig. 2. We observe that the number of constraints is generally between $0.6n$ and $0.7n$.

In the third experiment, we investigated the effect of the rate of the code on the performance of the algorithm. Fig. 3 shows the average and maximum numbers of parity-check constraints in the final iteration where the block length is $n = 120$ and the number of parity checks, m , increases from 15 to 90. The variable node degree is fixed at $d_v = 3$. We see that the average and maximum numbers of constraints are respectively in the ranges $1.1m$ to $1.2m$ and $1.4m$ to $1.6m$ for most values of m . The relatively large drop of the average

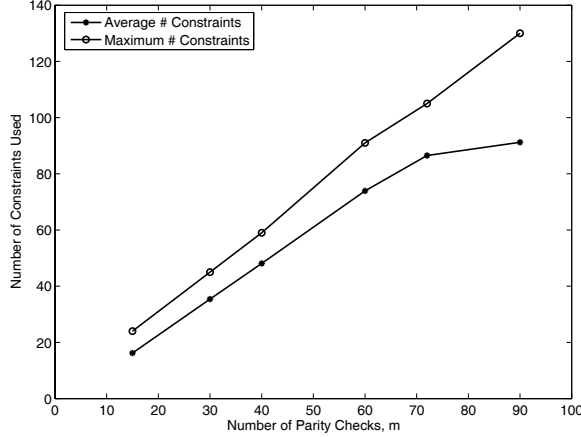


Fig. 3. The average and maximum number of parity-check constraints used versus the number of parity checks, m , for $n = 120$ and $d_v = 3$.

number for $m = 90$ with respect to the linear curve can be explained by the fact that at this value of m the rate of failure of LP decoding was less than 0.5 at -1.0 dB, whereas for all the other values of m , this rate was close to 1. Since the success of LP decoding generally indicates proximity of the received vector to a codeword, we expect the number of parity checks required to converge to be small in such a case, which decreases the average number of constraints.

As a demonstration of the gain in the running time of the adaptive algorithm, we observed that with a regular code of check degree 4 the adaptive method was about 12 times faster than the standard LP in the worst case, and this gain increased to about 120 with the check degree equal to 6. Based on the simulation results, we observe that in practice the algorithm performs much faster than is guaranteed by Theorem 2. These observations suggest the following conjecture.

Conjecture 1: For a random parity-check code of length n with m parity checks and arbitrary degree distributions, as n and m increase arbitrarily, the adaptive LP decoding algorithm converges to the optimal point of the fundamental polytope with high probability in at most α iterations and with at most βm final parity-check constraints, where α and β are constants independent of the SNR and the length, rate, and degree distribution of the code.

IV. GENERATING CUTS TO IMPROVE THE PERFORMANCE

The complexity reduction obtained by adaptive LP decoding inspires the use of cutting-plane techniques to improve the error rate performance of the algorithm. Specifically, when LP with all the original constraints gives a nonintegral solution, we try to cut the current solution, while keeping all the possible integral solutions (codewords) feasible.

In the decoding problem, the new cuts can be chosen from a pool of constraints describing a relaxation of the maximum-likelihood problem which is tighter than the fundamental polytope. In this sense, the cutting-plane technique is equivalent to the adaptive LP decoding of the previous section, with the difference that there are more constraints to choose from. The effectiveness of this method depends on how closely

the new relaxation approximates the ML decoding problem, and how efficiently we can search for those constraints that introduce cuts. Feldman *et al.* [1] have mentioned some ways to tighten the relaxation of the ML decoding, including adding redundant parity checks (RPC), and using lift-and-project methods. (For more on lift-and-project, see [6] and references therein.) Gomory's algorithm [7] is also one of the most well-known techniques for general integer optimization problems, although it suffers from slow convergence. Each of these methods can be applied adaptively in the context of cutting-plane techniques.

The simple structure of RPCs makes them an interesting choice for generating cuts. There are examples where even the relaxation obtained by adding all the possible RPC constraints does not guarantee convergence to a codeword. In other words, it is possible that we obtain a nonintegral solution for which there is no RPC cut, although the general case is still not well-studied. Also, finding efficient methods to search for RPC cuts for a given nonintegral solution is an open issue. On the other hand, as observed in simulation results, RPC cuts are generally strong, and a reasonable number of them makes the resulting LP relaxation tight enough to converge to an integer-valued solution. In this work, we focus on cutting-plane algorithms that use RPC cuts.

A. Finding Redundant Parity-Check Cuts

A redundant parity check is obtained by modulo-2 addition of some of the rows of the parity-check matrix, and this new check introduces a number of constraints that may include a cut. There is an exponential number of RPCs that can be made this way, and in general, most of them do not introduce cuts. Hence, we need to find the cuts efficiently by exploiting the particular structure of the decoding problem. In particular, we observe that cycles in the graph have an important role in determining whether an RPC generates a cut. This property is explained by Theorem 3.

Definition 2: Given a current solution, $\underline{x}$, the subset $T \subset \{1, 2, \dots, m\}$ of check node indices is called a *cut-generating collection* if the RPC made by modulo-2 addition of the parity checks corresponding to T introduces a cut.

Theorem 3: Let $T \subset \{1, 2, \dots, m\}$ be a collection of check node indices in the Tanner graph of the code, and let G be the subgraph made up of these check nodes, the variable nodes directly connected to them, and all the edges that connect them. Then, T can be a cut-generating collection only if G contains a cycle that only passes through variable nodes whose corresponding current values are fractional. (Proof omitted.)

This result motivates the following algorithm to search for cuts.

Algorithm 3:

Step 1: Having a solution $\underline{x}$, prune the Tanner graph by removing all the variable nodes with integer values.

Step 2: Starting from an arbitrary check node, randomly walk through the pruned graph until a cycle is found.

Step 3: Create an RPC by combining the rows of the parity-check matrix corresponding to the check nodes in the cycle.

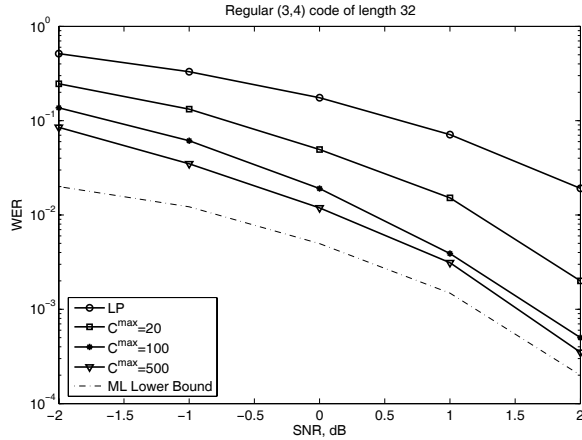


Fig. 4. WER of cutting-plane LP versus SNR for different values of C_{max} .

Step 4: If this RPC does not introduce a cut, go to Step 2.

Our experiments indicate that by exploiting some of the properties of the problem as described above, the search for each cut becomes faster by orders of magnitude than the naive search, in which RPCs are selected in a completely random way. However, this procedure is still the complexity bottleneck of the decoding, and it becomes prohibitively complex for codes of length on the order of several hundred bits and above. On the other hand, experiments demonstrate that the number of cuts required to converge to the ML codeword (if the convergence is possible) does not grow very rapidly with length. This motivates more study of the properties of RPC cuts and efficient schemes to find them.

B. Numerical Results

To demonstrate the performance improvement achieved by using the RPC cutting-plane technique, we present simulation results for a random regular (3, 4) LDPC code of length 32. The adaptive nature of the algorithm allows us to smoothly trade complexity for performance by changing the number of trials in the search for each RPC cut, i.e. the number of iterations in Algorithm 3, as well as the total number of calls of the LP solver. In this experiment, we fix the total number of iterations of Algorithm 3 to C_{max} , and declare decoding failure if no cut is found after C_{max} trials.

The word error rate (WER) of the algorithm is plotted versus SNR in Fig. 4 for different values of C_{max} . For comparison, the WER of pure LP decoding, i.e. with no RPC cut, and a lower bound on the WER of the ML decoder have been included, as well. In order to obtain this lower bound, we count the number of times that the cutting-plane LP algorithm with a large value of C_{max} converges to a codeword other than the transmitted codeword, and divide that by the number of blocks. Due to the ML certificate property of LP decoding, we know that ML decoding would fail in those cases, as well. On the other hand, ML decoding may also fail in some of the cases where LP decoding does not converge to an integral solution. Therefore, this estimate gives a lower bound on the WER of ML decoding.

The numerical results suggest that the cutting-plane LP decoding with RPC cuts can significantly outperform the pure LP decoding, at the cost of increased complexity.

V. CONCLUSION

In this work, we studied the potential for improving LP decoding, both in complexity and error correction capability, by using an adaptive approach. The key idea was to use the fact that we can always recognize the failure of LP decoding to find the ML codeword, a property that message-passing algorithms only have in specific cases such as the erasure channel. This feature allows us to add only constraints that are “useful”, depending on the current status of the algorithm.

In the algorithm proposed in Section III, the complexity is significantly reduced and becomes independent of the degree distributions, making it possible to apply LP decoding to parity-check codes of arbitrary densities. However, since general purpose LP solvers are used at each iteration, the complexity in terms of the block length is still super-linear, as opposed to linear as in the message-passing algorithms. An interesting question is whether we can design special LP solvers for decoding of LDPC codes that can take advantage of the sparsity of the constraints and other properties of the problem to converge in linear time.

Section IV serves as a first step to explore the application of cutting-plane techniques in LP decoding. A desirable feature of this approach is that by changing the parameters of the algorithm we can smoothly trade complexity for performance. In contrast, if we want to get the same performance gains by tightening the relaxation in a non-adaptive setting, the required complexity increases much faster. We showed that redundant parity checks provide strong cuts, even though they may not guarantee ML performance. A major open problem is to find efficient ways to search for these cuts by exploiting their properties, and to determine specific classes of codes for which RPC cuts are more effective. Furthermore, the effectiveness of cuts generated by other techniques, such as lift-and-project cuts and Gomory cuts, as well as specially designed cuts, needs further study.

REFERENCES

- [1] J. Feldman, M. J. Wainwright, and D. Karger, “Using linear programming to decode binary linear codes,” *IEEE Trans. Inform. Theory*, vol. 51, no. 3, pp. 954-972, Mar. 2005.
- [2] P. O. Vontobel and R. Koetter, “On the relationship between linear programming decoding and min-sum algorithm decoding,” *Proc. Int’l Symp. on Inform. Theory and Applications*, pp. 991-996, Parma, Italy, Oct. 2004.
- [3] P. O. Vontobel and R. Koetter, “Graph-cover decoding and finite-length analysis of message-passing iterative decoding of LDPC codes,” *Submitted to IEEE Trans. Inform. Theory*.
- [4] M. H. Taghavi N. and P. H. Siegel, “Adaptive linear programming decoding,” *in preparation*, a draft containing the proofs is available at <http://ieng9.ucsd.edu/~mthagavi/publications>.
- [5] “GNU Linear Programming Kit,” Available at <http://www.gnu.org/software/glpk>.
- [6] M. Laurent, “A comparison of the Sherali-Adams, Lovász-Schrijver and Lasserre relaxations for 0-1 programming,” *Mathematics of Operations Research*, vol. 28, no. 3, pp. 470-496, 2003.
- [7] R. Gomory, “An algorithm for integer solutions to linear programs,” *Princeton IBM Mathematical Research Report*, Nov. 1958.